

Prediction of Next Day Market Movement

Background Information

An Exchange Traded Fund (ETF) is a liquid investment vehicle that shares many characteristics of traditional stocks. It is generally a liquid asset, meaning it is easy to get in and out of positions; however, these assets are desirable because they typically track certain indices. For example, SPY is a State Street fund that tracks the S&P 500 index. But it doesn't have to track a stock index; there are also ETFs, such as IAU, and BlackRock's iShares Gold ETF, that allow investors to trade the price of gold. With this in mind, I am developing and backtesting a strategy to trade these ETFs daily, where the signal is derived from Machine Learning models trained on features often used in statistical methodologies for trading traditional stocks.

I sourced data using the YFinance library, which provides historical OHLC (open, high, low, and close) data for as long as an ETF has been trading. Additionally, I gathered their net asset value (NAV) and inception date from this package and stored them in a CSV. This data is split-adjusted and does not include dividend value. I chose that because it is historically the closest to what was actually traded that day.

In my project, I am looking at 74 of the 100 largest ETFs by NAV. I chose these specific ETFs because they have enough training data and will provide the most liquidity should this become a viable trading solution. In the prototype, I built a correlation matrix of returns over the entire period and then used DBSCAN to identify the clusters each ETF belongs to. DBSCAN identified some ETFs that didn't belong to a group, but upon further research, I placed them in their respective groups, which I noted in the Jupyter Notebook. In my prototype assignment, I planned to tune hyperparameters based on clusters; however, when I worked with XGBoost, all the groups tuned similarly, and if I became any more aggressive, there was serious overfitting. Additionally, since these ETFs were treated the same in terms of technical features, there wasn't anything specific to that ETF sector. I found that the models would learn.

I have focused a lot of energy on engineering the proper features. The features include, K which is a stochastic property that looks at current price vs recent range, D - signal line for momentum, K_minus_D - spread between K and D, and shows momentum strength/direction, K_cross_D - a momentum inflection point, dist_ma - which is the distance from the moving average showing a deviation from the mean, vol_ratio which is the relative volatility vs a 200-day baseline, and Standardized Regime - a normalized market state label for a particular asset (e.g., bull market with high volatility). Given these features for each fund, each model was tested for overfitting by checking the accuracy of the training and testing data. Each model has similar accuracy, indicating no overfitting or data leakage. This is then used as a signal to determine the long/short positions. If the probability is >55%, the model goes long; if it is <45%, the model goes short.

Models Used

As noted, I used Logistic Regression, XGBoost, and additionally an LSTM Recurrent Neural Network. Since the overall goal of my project was to generate a probability as my trading signal, I instinctively chose logistic regression as a baseline. This model provides interpretable classification probabilities, which is exactly my goal. The training and testing datasets were split chronologically to preserve the time series nature of stock prices. For all scaling, I used StandardScaler from the sklearn package.

The second model implemented was XGBoost, a gradient-boosting algorithm that captures non-linear patterns in data. This model did a good job of capturing these movements between features; however, as the tree grew deeper, it became increasingly susceptible to overfitting. Keeping the tree models to a max depth of 4 produced the best results without overfitting. Generally, the XGBoost models produced stronger classification confidence and generated more trades.

Finally, where I spent most of my time was developing and training a Long Short-Term Memory recurrent neural network. LSTM RNNs are designed for sequential (time series) data. Initially, when I started in mid-April, I hadn't considered using this model, but after learning about it in class and the book,

I realized it was exactly what I was looking for. Financial markets contain temporary and long-term relationships in price behavior, which is reflected in my stochastic features. The LSTM model is designed to retain important information and identify key patterns in sequences, which aligns with the theory that financial markets exhibit a memory-like pattern. I used the PyTorch package to implement the models. I used [the GeeksforGeeks](#) and [PyTorch](#) documentation to implement the LSTM model. I mirrored my sequence method from Geeks for Geeks and had a fixed look-back period of 20 days, which is roughly a trading month. Just as with Logistic Regression and XGBoost, LSTM was used to generate probabilities for next-day classification, which were then used as my trade signals.

Since the dataset was rather small for each LSTM model, the number of layers and hidden size were optimal at 16 and 1, respectively. 2 also worked for the hidden layer. Still, the model was overly confident that every day was going to be an up day, so I found it better performance-wise to use these as hyperparameters. To check for overfitting, I separated the data into training, validation, and test sets. I printed the training and validation losses and found they generally decreased together, suggesting little overfitting. However, this was not always the case, and, in retrospect, it would've been better to plot each model's loss function and determine which showed the most overfitting. Another issue I ran into was long training times across the entire dataset on my MacBook Air, so I had to train the LSTM model in batches. Additionally, when I first ran the model, it hung in the first epoch, so I had to look at the [PyTorch Forum](#) to realize I needed to specify running it on the CPU with a single thread.

Results

Overall, I am happy with the results. Would this be a good production model? Absolutely not. I did not study the number of trades per day; there was no rhyme or reason behind my position sizing besides a straight average. So, if one ETF signaled that day, the portfolio would be 100% on either the long or short side. Essentially, there was nothing done to mitigate risk. What I achieved here was the ability to use different algorithms to classify good bets and achieve returns more stable than those of the S&P 500. Additionally, this model does not consider factors such as being short on a day a dividend is

paid, economic data, or the trading costs of moving in and out of a position each day, which, over time, will destroy your returns. Unless these models were generating serious alpha (i.e., gains > the S&P 500 return or any benchmark), I would be unwilling to trade them myself.

To build the graphs used in the Jupyter notebook, I based them on a portion of this [video](#), which was a great inspiration for how I approached this project.

The reasons I am happy with the performance of the four backtested strategies are that, in every case, the Sharpe and Sortino ratios are higher than the benchmark. This was a goal of mine, which I achieved. These ratios measure risk-adjusted returns: the Sharpe ratio measures excess return per unit of total volatility, while the Sortino ratio measures excess return per unit of downside volatility. Across each strategy, the max drawdown is lower than the benchmark, and overall returns are fairly stable. One thing to note: in the Logistic Regression, XGBoost, and Average models, the max drawdown occurred on Donald Trump's "Liberation Day", which was unexpected. However, the benchmark fell again the following day, and the models I produced rebounded the next day, which suggests something about the strength of the features and their stochastic properties. Finally, as shown in the last cell, the Logistic Regression had the highest return, but of all the strategies I produced, it had the worst risk-adjusted returns. I believe the best strategy was to average the probabilities across all three models. It had a return similar to the benchmark, a better Sharpe ratio, and a significantly better Sortino ratio, which, if marketed to a retirement account or retiree looking for stable returns, would be a viable option.

Future Work

Beyond what I have mentioned about the technical aspects of this becoming a viable trading model, there are a few things that I could do to improve this project in the future. One of the most valuable features to add is getting constituent data on ETFs backed by stocks, essentially doing what I did here for each stock, and using the weighted-average probabilities as a feature to determine whether the ETF will go up. Additionally, I can strengthen my regime feature by using the Hurst exponent, which indicates whether the asset is mean-reverting. Also, adding macroeconomic data will help identify how an

ETF would move since it is more sensitive to broader market changes. Finally, I wanted to add the probabilities from Logistic Regression and XGBoost to the LSTM RNN initially. Then, I realized that the only probabilities I had were from the test data. It would be interesting to use a trained model to incorporate those probabilities into the LSTM, since it can learn sequences where Logistic Regression is confident for a few days in a row, then the fourth day could flip, or vice versa. Finally, if I added all of these features to the models, I would probably want to use Sequential Backward Selection to determine which features are actually important to reduce complexity. Overall, the project was a strong foundation for future research on quantitative trading systems and the applicability of machine learning in finance.